7th Open-Source Computational Mechanics and Acoustics Workshop and Summer Camp

Workshop 7: Getting the Topology Optimization Cycle into Work

Prof. Dr. Emre Alpman

General Optimization Problem

- A single objective optimization problem aims to minimize (or maximize) a scalar objective function;

$$minimize\, f(\vec{x})$$
$$st\,;\vec{h}(\vec{x})=0$$
$$\vec{g}(\vec{x})<0$$

- Where:
  - f is the objective function
  - $\vec{x}$ contains design variables
  - $\vec{h}(\vec{x}),\ \vec{g}(\vec{x})$ are equality and inequality constraint functions

https://mdobook.github.io/
https://dafoam.github.io/user-guide-background.html

Iterative Optimization Process

- Starting from and initial design; $\vec{x}_0$

$$\vec{x}_{k+1} = \vec{x}_k + \alpha_k \vec{d}_k \qquad k = 0, 1, 2, \ldots$$

- Here, α_k is the step size, and $\vec{d}_k$ is the search direction at step *k*.
- For many engineering problems the objective function can not be directly computed using the design variables.
  - It may be related to state variables $\vec{w}$ as well as design variables $\vec{x}$; $f\left(\vec{w}, \vec{x}\right)$
  - For example drag force (objective function) depends on the geometric parameters (design variables) and flow variables (state variables).

https://mdobook.github.io/
https://dafoam.github.io/user-guide-background.html

Iterative Optimization Process

- In that case one needs to solve the governing equation(s) to obtain the state variables corresponsing to design variables
  - e.g. Navier-Stokes equation for a problem involving fluid flow

$$\vec{R}(\vec{w},\vec{x})=\vec{0}$$

- Procedure: For a given $\vec{x}$
  - Solve the governing equations to obtain state variables
  - Calculate the objective function $f(\vec{w},\vec{x})$

https://mdobook.github.io/
https://dafoam.github.io/user-guide-background.html

Iterative Optimization Process

- In that case one needs to solve the governing equation(s) to obtain the state variables corresponsing to design variables
  - e.g. Navier-Stokes equation for a problem involving fluid flow

$$\vec{R}(\vec{w},\vec{x})=\vec{0}$$

- Procedure: For a given $\vec{x}$
  - Solve the governing equations to obtain state variables
    - Typically requires numerical solution (CFD solution)
  - Calculate the objective function $f(\vec{w},\vec{x})$
    - Might require integration or differentiation

https://mdobook.github.io/
https://dafoam.github.io/user-guide-background.html

Search Direction

- In gradient based methods search direction is found using the gradient of the objective function (assumed to be scalar) with respect to the design variables;

$$\nabla f = \frac{df}{d\vec{x}}$$

- Steepest descent method: $\vec{d} = -\frac{\nabla f}{|\nabla f|}$

- More advanced methods*: Sequential Quadratic Programming, Interior Point Methods, Augmented Lagrangian Methods

* https://mdobook.github.io/

Gradient Calculation

- Many times gradient of the objective function can not be obtained analytically.
  - Numerical differentiation is applied.
- For problems involving a large number of design variables, gradient computation may be prohibitively expensive.
  - Algorithmic differentiation
    - Forward mode
    - Reverse mode (adjoint differentiation)

Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey. Journal of machine learning research, 18(153), 1-43.

Algorithmic Differentiation

- Automatic Differentiation approach computes gradients by factoring the operation into a series of elementary operations whose derivatives are known.
- Overall gradient is computed by systematically applying chain rule.
- Computational cost is typically proportional to number of design variables.
- At elementary level differentiation is performed symbolically.
  - The resulting derivatives are exact up to machine precision.

Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey. Journal of machine learning research, 18(153), 1-43.

Algorithmic Differentiation: Forward Mode

- Conceptually the simplest approach.
- A given functional relation is constructed by defining intermediate variables.
- Let the function $f : \Re^n \rightarrow \Re^m$
- With $x_i ; i = 1, \ldots, n$ are the design variables and

  $y_i ; i = 1, \ldots, m$ are the output values
- We define:

$$v_{i-n} = x_i \qquad i = 1, \ldots, n$$

$$v_{l-i} = y_i \qquad i = 1, \ldots, m$$

$$v_i \qquad i = 1, \ldots, l$$ Intermediate variables

Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey. Journal of machine learning research, 18(153), 1-43.

Algorithmic Differentiation: Forward Mode

- In order to compute $\frac{\partial f}{\partial x_k}$:
- We differentiate each intermediate variable: $\dot{v}_i = \frac{\partial v_i}{\partial x_k}$
- Since the output values are also represented by intermediate variables, derivative we seek will eventually be obtained.

Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey. Journal of machine learning research, 18(153), 1-43.

Algorithmic Differentiation: Forward Mode

- Example
- Let $y = f(x_1, x_2) = \ln\left(\frac{x_1 x_2}{\sqrt{x_1^2 + x_2^2}}\right)$
- Calculate $\frac{\partial y}{\partial x_1}$ at $(x_1, x_2) = (2, 3)$
- We can write:

$$v_{-1} = x_1 = 2$$
$$v_0 = x_2 = 3$$
$$v_1 = v_0 v_{-1} = 6$$
$$v_2 = v_0^2 + v_{-1}{}^2 = 13$$
$$v_3 = \sqrt{v_2} = 3.606$$
$$v_4 = \frac{v_1}{v_3} = 1.664$$
$$v_5 = \ln(v_4) = y = 0.509$$

Algorithmic Differentiation: Forward Mode

- For $\frac{\partial y}{\partial x_1}$ we let $\dot{v}_{-1}=1$ and $\dot{v}_0=0$

- Then

$$\dot{v}_{-1}=1$$

$$\dot{v}_0=0$$

$$\dot{v}_1=\dot{v}_0 v_{-1}+v_0 \dot{v}_{-1}=v_0=3$$

$$\dot{v}_2=2 v_0 \dot{v}_0+2 v_{-1} \dot{v}_{-1}=2 v_{-1}=4$$

$$\dot{v}_3=\frac{1}{2\sqrt{v_2}}\dot{v}_2=\frac{4}{2\sqrt{13}}=0.555$$

$$\dot{v}_4=\frac{\dot{v}_1 v_3-v_1 \dot{v}_3}{v_3^2}=\frac{(3)(3.606)-(6)(0.555)}{3.606^2}0.576$$

$$\dot{v}_5=\frac{\dot{v}_4}{v_4}=\frac{\partial y}{\partial x_1}=\frac{0.576}{1.664}=0.346$$

Algorithmic Differentiation: Forward Mode

- Directional derivative $\frac{\partial y}{\partial n} = \nabla y \cdot \vec{n}$
- Knowing that

$$\frac{\partial y}{\partial x_1} = \nabla y \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad \frac{\partial y}{\partial x_2} = \nabla y \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

- For a given $\vec{n} = \begin{bmatrix} n_1 \\ n_2 \end{bmatrix}$

- Directional derivative can be computed with forward mode simply by specifiying:

$$\dot{v}_{-1} = n_1 \qquad \dot{v}_0 = n_2$$

Method is also suitable for computing the dot product of a vector with the Jacobian matrix

Algorithmic Differentiation: Reverse Mode

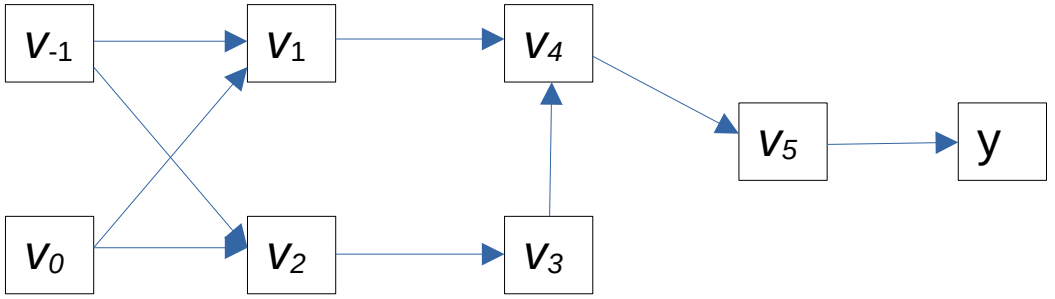
- Recall: $f : \Re^n \rightarrow \Re^m$
- Reverse (or adjoint) mode is preferred when $n >> m$.
- Reverse mode consists of two phases:
  - In the first phase intermediate variables are defined by propagating in forward direction.
  - In the second phase derivatives are calculating by propagating in the reverse direction
- Here we define the adjoint: $\bar{v}_i = \frac{\partial y}{\partial v_i}$
- In order to compute adjoints we resort to the computational graph.

Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey. Journal of machine learning research, 18(153), 1-43.

Algorithmic Differentiation: Reverse Mode

- Let $y = f(x_1, x_2) = \ln\left(\frac{x_1 x_2}{\sqrt{x_1^2 + x_2^2}}\right)$
- Calculate $\frac{\partial y}{\partial x_1}$ at $(x_1, x_2) = (2, 3)$
- We can write:

$$v_{-1} = x_1 = 2$$
$$v_0 = x_2 = 3$$
$$v_1 = v_0 v_{-1} = 6$$
$$v_2 = v_0^2 + v_{-1}{}^2 = 13$$
$$v_3 = \sqrt{v_2} = 3.606$$
$$v_4 = \frac{v_1}{v_3} = 1.664$$
$$v_5 = \ln(v_4) = y = 0.509$$



Computational Graph

Algorithmic Differentiation: Reverse Mode

- Starting from the last intermediate variable:

$$\bar{v}_5 = \frac{\partial y}{\partial v_5} = 1$$

$$\bar{v}_4 = \frac{\partial y}{\partial v_4} = \frac{\partial y}{\partial v_5}\frac{\partial v_5}{\partial v_4} = \bar{v}_5 \frac{1}{v_4}$$

$$\bar{v}_3 = \frac{\partial y}{\partial v_3} = \frac{\partial y}{\partial v_4}\frac{\partial v_4}{\partial v_3} = -\bar{v}_4 \frac{v_1}{v_3^2}$$

$$\bar{v}_2 = \frac{\partial y}{\partial v_2} = \frac{\partial y}{\partial v_3}\frac{\partial v_3}{\partial v_2} = \bar{v}_3 \frac{1}{2\sqrt{v_2}}$$

$$\bar{v}_1 = \frac{\partial y}{\partial v_1} = \frac{\partial y}{\partial v_4}\frac{\partial v_4}{\partial v_1} = \bar{v}_4 \frac{1}{v_3}$$

$$\bar{v}_0 = \frac{\partial y}{\partial v_0} = \frac{\partial y}{\partial v_1}\frac{\partial v_1}{\partial v_0} + \frac{\partial y}{\partial v_2}\frac{\partial v_2}{\partial v_0} = \bar{v}_1 v_{-1} + \bar{v}_2 2 v_0$$

$$\bar{v}_{-1} = \frac{\partial y}{\partial v_{-1}} = \frac{\partial y}{\partial v_1}\frac{\partial v_1}{\partial v_{-1}} + \frac{\partial y}{\partial v_2}\frac{\partial v_2}{\partial v_{-1}} = \bar{v}_1 v_0 + \bar{v}_2 2 v_{-1}$$

Algorithmic Differentiation: Reverse Mode

- Starting from the last intermediate variable:

$$\bar{v}_5 = \frac{\partial y}{\partial v_5} = 1$$

$$\bar{v}_4 = \frac{\partial y}{\partial v_4} = \frac{\partial y}{\partial v_5}\frac{\partial v_5}{\partial v_4} = \bar{v}_5 \frac{1}{v_4} = 0.601$$

$$\bar{v}_3 = \frac{\partial y}{\partial v_3} = \frac{\partial y}{\partial v_4}\frac{\partial v_4}{\partial v_3} = -\bar{v}_4 \frac{v_1}{v_3^2} = -0.277$$

$$\bar{v}_2 = \frac{\partial y}{\partial v_2} = \frac{\partial y}{\partial v_3}\frac{\partial v_3}{\partial v_2} = \bar{v}_3 \frac{1}{2\sqrt{v_2}} = -0.0384$$

$$\bar{v}_1 = \frac{\partial y}{\partial v_1} = \frac{\partial y}{\partial v_4}\frac{\partial v_4}{\partial v_1} = \bar{v}_4 \frac{1}{v_3} = 0.167$$

$$\bar{v}_0 = \frac{\partial y}{\partial v_0} = \frac{\partial y}{\partial v_1}\frac{\partial v_1}{\partial v_0} + \frac{\partial y}{\partial v_2}\frac{\partial v_2}{\partial v_0} = \bar{v}_1 v_{-1} + \bar{v}_2 2 v_0 = 0.103$$

$$\bar{v}_{-1} = \frac{\partial y}{\partial v_{-1}} = \frac{\partial y}{\partial v_1}\frac{\partial v_1}{\partial v_{-1}} + \frac{\partial y}{\partial v_2}\frac{\partial v_2}{\partial v_{-1}} = \bar{v}_1 v_0 + \bar{v}_2 2 v_{-1} = 0.346$$

Algorithmic Differentiation: Reverse Mode

- Starting from the last intermediate variable:

$$\bar{v}_5 = \frac{\partial y}{\partial v_5} = 1$$

$$\bar{v}_4 = \frac{\partial y}{\partial v_4} = \frac{\partial y}{\partial v_5}\frac{\partial v_5}{\partial v_4} = \bar{v}_5 \frac{1}{v_4} = 0.601$$

$$\bar{v}_3 = \frac{\partial y}{\partial v_3} = \frac{\partial y}{\partial v_4}\frac{\partial v_4}{\partial v_3} = -\bar{v}_4 \frac{v_1}{v_3^2} = -0.277$$

$$\bar{v}_2 = \frac{\partial y}{\partial v_2} = \frac{\partial y}{\partial v_3}\frac{\partial v_3}{\partial v_2} = \bar{v}_3 \frac{1}{2\sqrt{v_2}} = -0.0384$$

$$\bar{v}_1 = \frac{\partial y}{\partial v_1} = \frac{\partial y}{\partial v_4}\frac{\partial v_4}{\partial v_1} = \bar{v}_4 \frac{1}{v_3} = 0.167$$

$$\bar{v}_0 = \frac{\partial y}{\partial v_0} = \frac{\partial y}{\partial v_1}\frac{\partial v_1}{\partial v_0} + \frac{\partial y}{\partial v_2}\frac{\partial v_2}{\partial v_0} = \bar{v}_1 v_{-1} + \bar{v}_2 2 v_0 = 0.103$$

$$\bar{v}_{-1} = \frac{\partial y}{\partial v_{-1}} = \frac{\partial y}{\partial v_1}\frac{\partial v_1}{\partial v_{-1}} + \frac{\partial y}{\partial v_2}\frac{\partial v_2}{\partial v_{-1}} = \bar{v}_1 v_0 + \bar{v}_2 2 v_{-1} = 0.346$$

The gradient of the function at the specified points is obtained in one sweep

Adjoint Approach for Optimization

- Recall:
- *For many engineering problems the objective function can not be directly computed using the design variables*.
- Governing equation: $\vec{R}(\vec{w},\vec{x})=\vec{0}$
- Objective Function (assumed to be scalar): $f(\vec{w},\vec{x})$
- Total derivative of objective function:

$$df=\frac{\partial f}{\partial \vec{x}}d\vec{x}+\frac{\partial f}{\partial \vec{w}}d\vec{w}$$

$$\frac{df}{d\vec{x}}=\frac{\partial f}{\partial \vec{x}}+\frac{\partial f}{\partial \vec{w}}\frac{d\vec{w}}{d\vec{x}}$$

Kenway, G. K., Mader, C. A., He, P., & Martins, J. R. (2019). Effective adjoint approaches for computational fluid dynamics. Progress in Aerospace Sciences, 110, 100542.

Adjoint Approach for Optimization

- Recall:
- *For many engineering problems the objective function can not be directly computed using the design variables*.
- Governing equation: $\vec{R}(\vec{w},\vec{x})=\vec{0}$
- Objective Function (assumed to be scalar): $f(\vec{w},\vec{x})$
- Total derivative of objective function:

$$df=\frac{\partial f}{\partial \vec{x}}d\vec{x}+\frac{\partial f}{\partial \vec{w}}d\vec{w}$$

$$\frac{df}{d\vec{x}}=\frac{\partial f}{\partial \vec{x}}+\frac{\partial f}{\partial \vec{w}}\frac{d\vec{w}}{d\vec{x}}$$

Kenway, G. K., Mader, C. A., He, P., & Martins, J. R. (2019). Effective adjoint approaches for computational fluid dynamics. Progress in Aerospace Sciences, 110, 100542.

Adjoint Approach for Optimization

- From the governing equation set: $\vec{R}(\vec{w},\vec{x})=\vec{0}$

$$\frac{d\vec{R}}{d\vec{x}}=\frac{\partial\vec{R}}{\partial\vec{x}}+\frac{\partial\vec{R}}{\partial\vec{w}}\frac{d\vec{w}}{d\vec{x}}$$

- The governing equations should not be affected by the change in design variables:

$$\frac{d\vec{R}}{d\vec{x}}=\frac{\partial\vec{R}}{\partial\vec{x}}+\frac{\partial\vec{R}}{\partial\vec{w}}\frac{d\vec{w}}{d\vec{x}}=0$$

$$\frac{d\vec{w}}{d\vec{x}}=-\left(\frac{\partial\vec{R}}{\partial\vec{w}}\right)^{-1}\frac{\partial\vec{R}}{\partial\vec{x}}$$

Kenway, G. K., Mader, C. A., He, P., & Martins, J. R. (2019). Effective adjoint approaches for computational fluid dynamics. Progress in Aerospace Sciences, 110, 100542.

Adjoint Approach for Optimization

- Submitting into

$$\frac{df}{d\vec{x}} = \frac{\partial f}{\partial \vec{x}} + \frac{\partial f}{\partial \vec{w}} \frac{d\vec{w}}{d\vec{x}}$$

$$\frac{df}{d\vec{x}} = \frac{\partial f}{\partial \vec{x}} - \frac{\partial f}{\partial \vec{w}} \left(\frac{\partial \vec{R}}{\partial \vec{w}} \right)^{-1} \frac{\partial \vec{R}}{\partial \vec{x}}$$

- Define: $\vec{\psi}^T = \frac{\partial f}{\partial \vec{w}} \left(\frac{\partial \vec{R}}{\partial \vec{w}} \right)^{-1}$ $\Rightarrow$ $\left(\frac{\partial \vec{R}^T}{\partial \vec{w}} \right) \vec{\psi} = \left(\frac{\partial f}{\partial \vec{w}} \right)^T$

Adjoint Equation

$$\frac{df}{d\vec{x}} = \frac{\partial f}{\partial \vec{x}} - \vec{\psi}^T \frac{\partial \vec{R}}{\partial \vec{x}}$$

Kenway, G. K., Mader, C. A., He, P., & Martins, J. R. (2019). Effective adjoint approaches for computational fluid dynamics. Progress in Aerospace Sciences, 110, 100542.

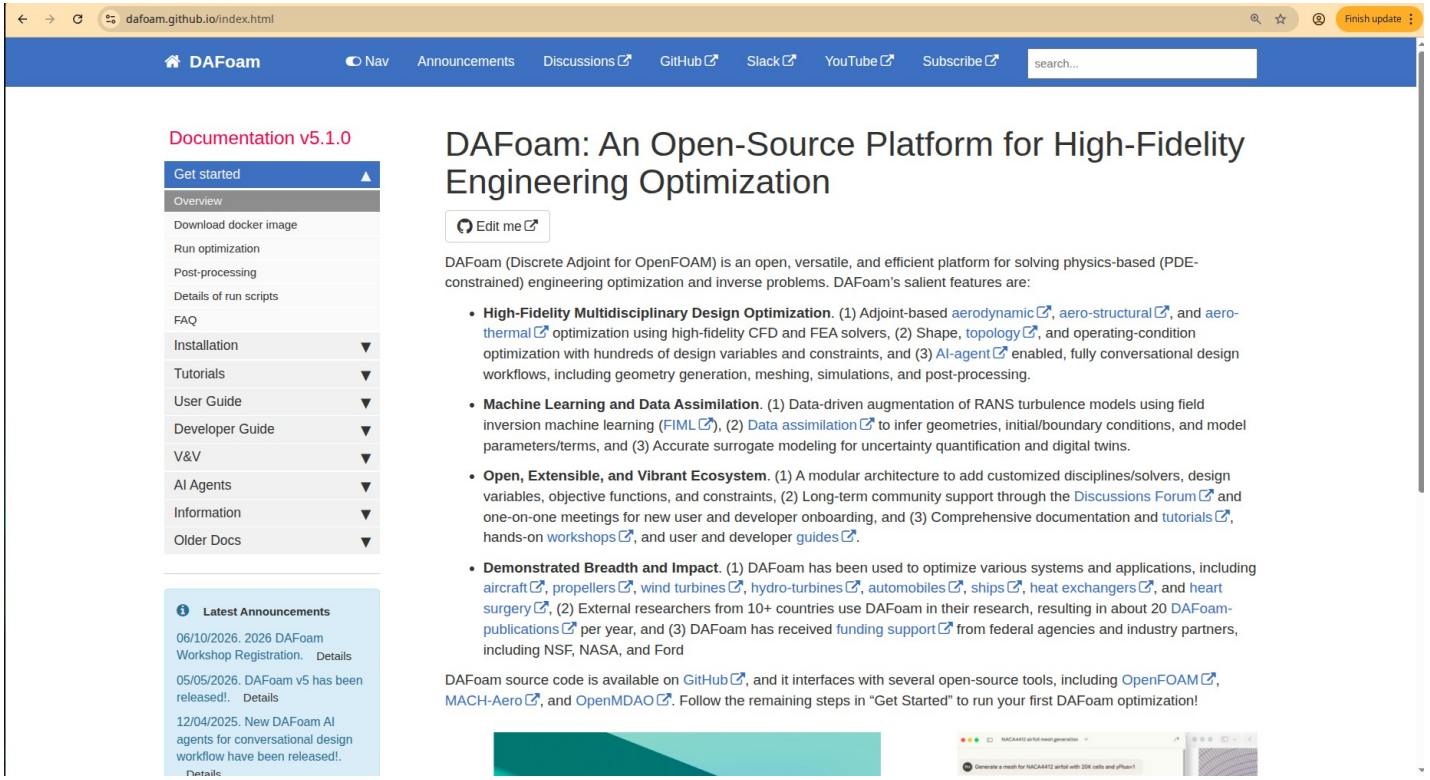
Adjoint Approach for Optimization

- Continous vs. Discrete Adjoint Approach
- Derivatives relative to state variables are calculated using the continuous or discrete form of the governing equations.
  - Nadarajah, S., & Jameson, A. (2000, January). A comparison of the continuous and discrete adjoint approach to automatic aerodynamic optimization. In 38th Aerospace sciences meeting and exhibit (p. 667).
- Partial derivatives with respect to design variables are typically obtained using perturbation techniques.

Kenway, G. K., Mader, C. A., He, P., & Martins, J. R. (2019). Effective adjoint approaches for computational fluid dynamics. Progress in Aerospace Sciences, 110, 100542.

DAFoam

- Disclaimer: This offering is not approved or endorsed by the developers of DAFoam (MDO Lab; https://mdolab.engin.umich.edu/)
- DAFoam: An Open-Source Platform for High-Fidelity Engineering Optimization
  - (https://dafoam.github.io/index.html)



Installation

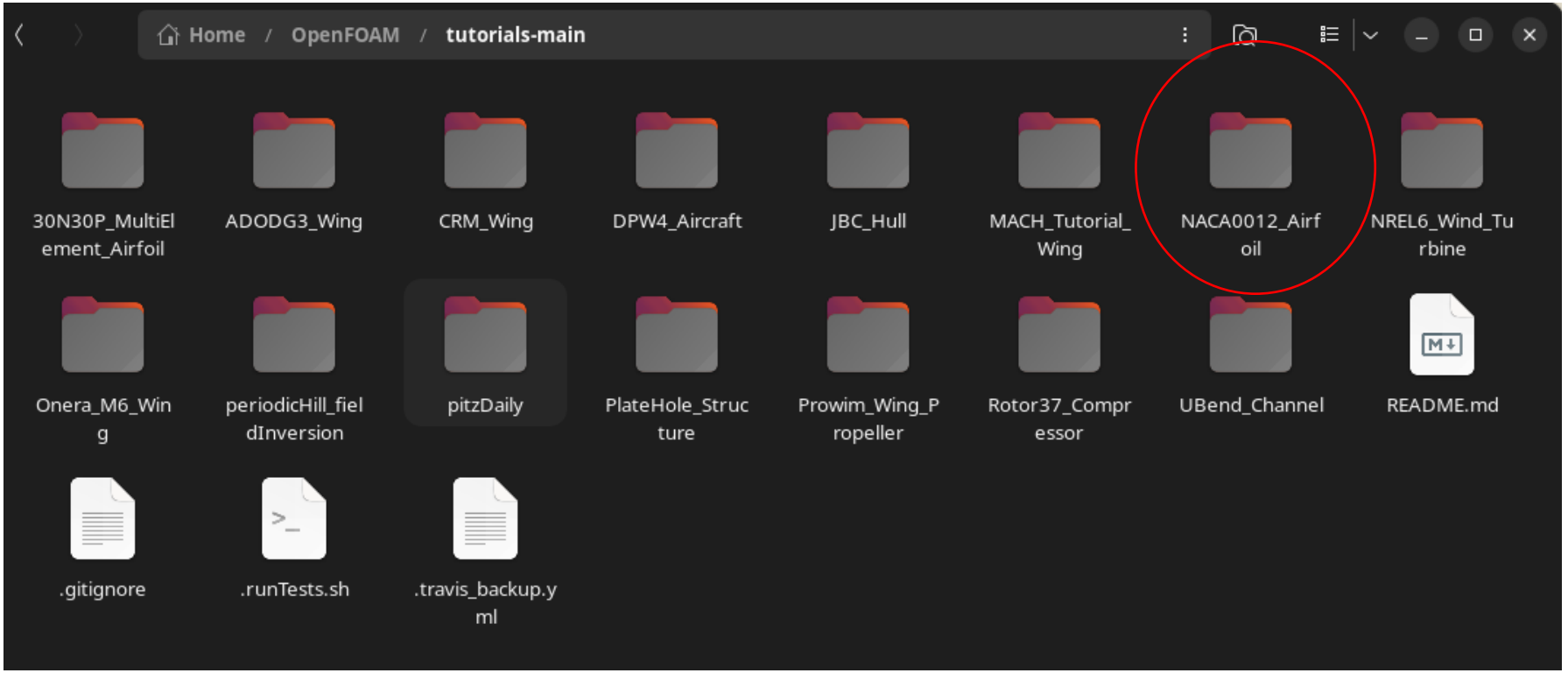
- There are two main alternatives:
  - Compile the source code
  - Build Docker images
- We will use Docker images.
- The operating system is Ubuntu 24.04
- https://docs.docker.com/desktop/setup/install/linux/ubuntu/

Installation

- One needs to pull the docker image:

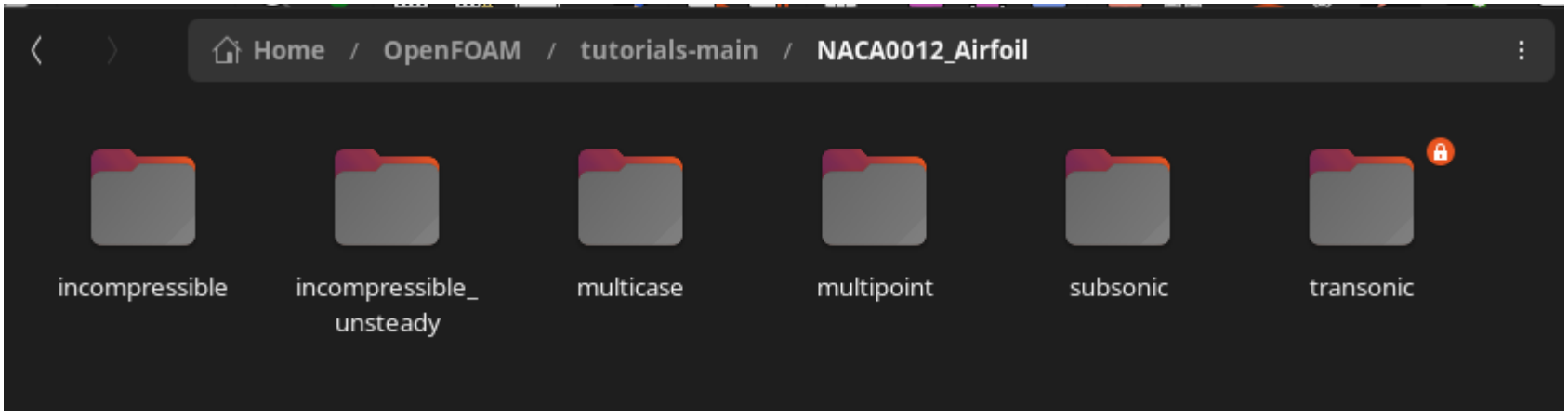
  >> **docker pull dafoam/opt-packages:v5.1.0**

- You can download the tutorials forlder from:
  - https://github.com/DAFoam/tutorials/archive/main.tar.gz
- Expand the compressed file to a suitable folder.



Example

- We will look into NACA0012 airfoil case
- The contents of the folders may be read only



- You may need to change the ownership of the files

```
emrealpman@eagle: ~/OpenFOAM/tutorials-main/NACA0012_Airfoil
emrealpman@eagle:~/OpenFOAM/tutorials-main/NACA0012_Airfoil$ ls -l
total 16
drwxrwxr-x 8 1002 1002 4096 Eyl  2 11:23 incompressible
drwxrwxr-x 7 1002 1002 4096 Eyl 22  2022 multipoint
drwxrwxr-x 7 1002 1002 4096 Eyl 22  2022 subsonic
drwxrwxr-x 7 1002 1002 4096 Eyl 22  2022 transonic
emrealpman@eagle:~/OpenFOAM/tutorials-main/NACA0012_Airfoil$
```

Example

- You may need to change the ownership of the files

```
emrealpman@eagle: ~/OpenFOAM/tutorials-main/NACA0012_Airfoil
emrealpman@eagle:~/OpenFOAM/tutorials-main/NACA0012_Airfoil$ ls -l
total 16
drwxrwxr-x 8 1002 1002 4096 Eyl  2 11:23 incompressible
drwxrwxr-x 7 1002 1002 4096 Eyl 22  2022 multipoint
drwxrwxr-x 7 1002 1002 4096 Eyl 22  2022 subsonic
drwxrwxr-x 7 1002 1002 4096 Eyl 22  2022 transonic
emrealpman@eagle:~/OpenFOAM/tutorials-main/NACA0012_Airfoil$ sudo chown emrealpm
an:emrealpman *
[sudo] password for emrealpman:
emrealpman@eagle:~/OpenFOAM/tutorials-main/NACA0012_Airfoil$ ls -l
total 16
drwxrwxr-x 8 emrealpman emrealpman 4096 Eyl  2 11:23 incompressible
drwxrwxr-x 7 emrealpman emrealpman 4096 Eyl 22  2022 multipoint
drwxrwxr-x 7 emrealpman emrealpman 4096 Eyl 22  2022 subsonic
drwxrwxr-x 7 emrealpman emrealpman 4096 Eyl 22  2022 transonic
emrealpman@eagle:~/OpenFOAM/tutorials-main/NACA0012_Airfoil$
```

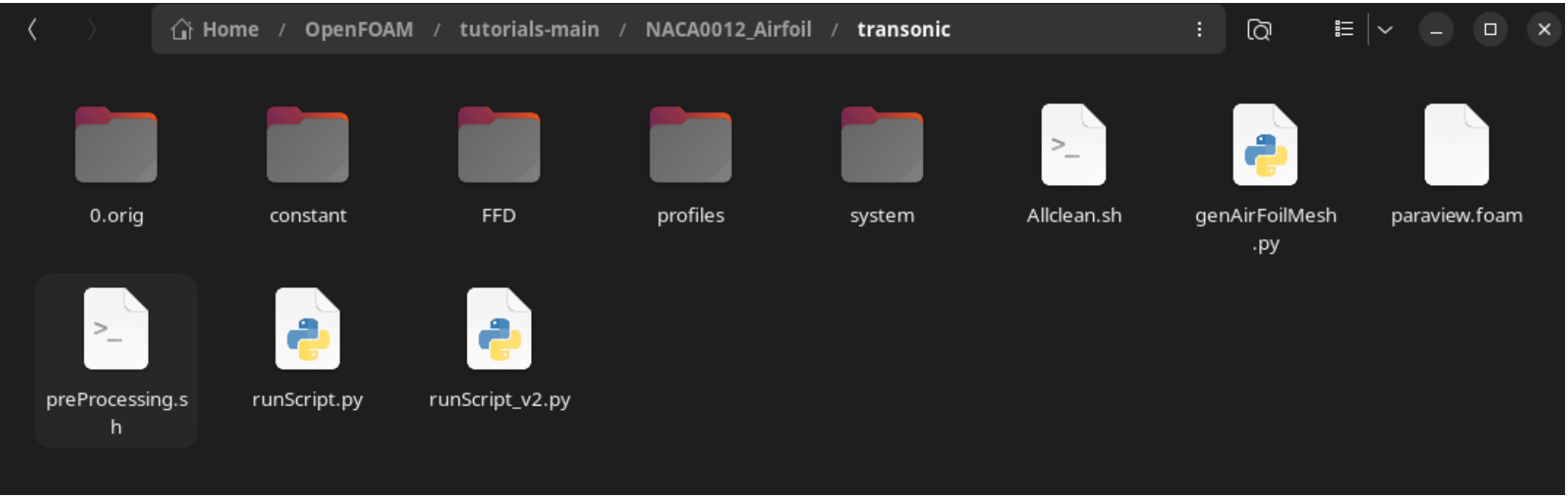
Example

- You may need to change the ownership of the files

```
emrealpman@eagle: ~/OpenFOAM/tutorials-main/NACA0012_Airfoil
emrealpman@eagle:~/OpenFOAM/tutorials-main/NACA0012_Airfoil$ ls -l
total 16
drwxrwxr-x 8 1002 1002 4096 Eyl  2 11:23 incompressible
drwxrwxr-x 7 1002 1002 4096 Eyl 22  2022 multipoint
drwxrwxr-x 7 1002 1002 4096 Eyl 22  2022 subsonic
drwxrwxr-x 7 1002 1002 4096 Eyl 22  2022 transonic
emrealpman@eagle:~/OpenFOAM/tutorials-main/NACA0012_Airfoil$ sudo chown emrealpm
an:emrealpman *
[sudo] password for emrealpman:
emrealpman@eagle:~/OpenFOAM/tutorials-main/NACA0012_Airfoil$ ls -l
total 16
drwxrwxr-x 8 emrealpman emrealpman 4096 Eyl  2 11:23 incompressible
drwxrwxr-x 7 emrealpman emrealpman 4096 Eyl 22  2022 multipoint
drwxrwxr-x 7 emrealpman emrealpman 4096 Eyl 22  2022 subsonic
drwxrwxr-x 7 emrealpman emrealpman 4096 Eyl 22  2022 transonic
emrealpman@eagle:~/OpenFOAM/tutorials-main/NACA0012_Airfoil$
```

Example

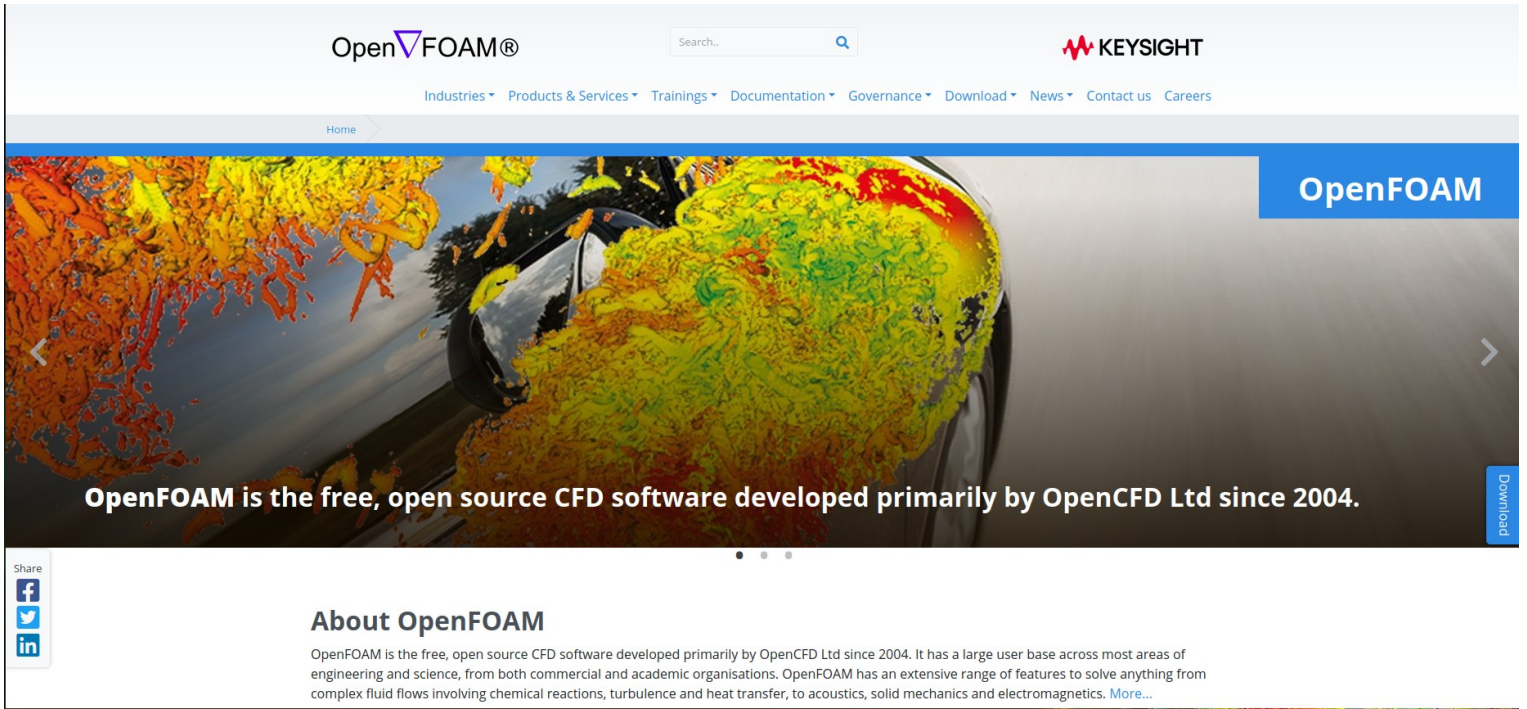
- Contents of the folder



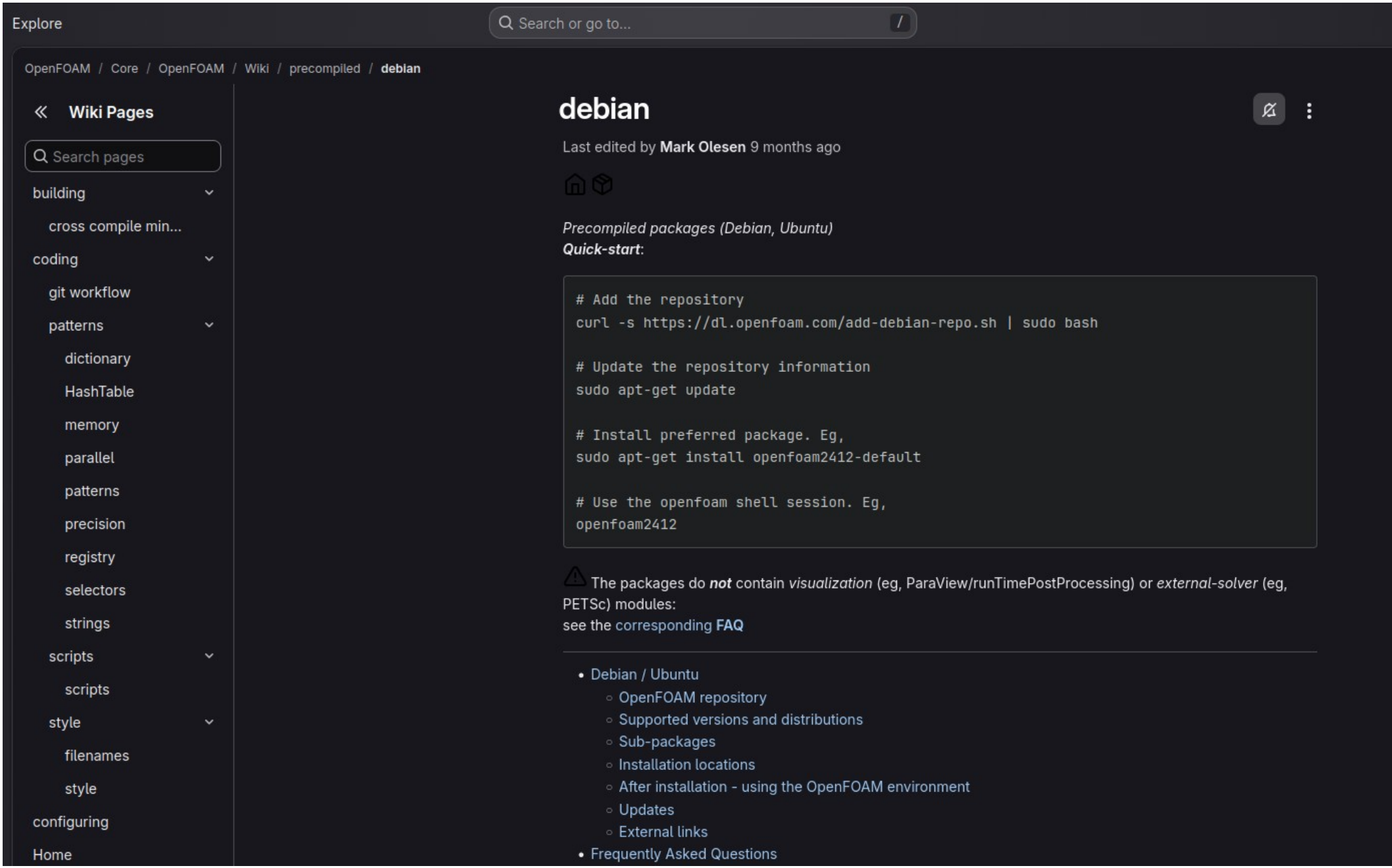
- DAFoam uses OpenFOAM® for solving the governing equations.
- OpenFOAM® cases consist of three main folders; *0*, *constant* and *system*

OpenFOAM®

- OpenFOAM® is a trade mark of OpenCFD Ltd.
- Disclaimer: This offering is not approved or endorsed by OpenCFD Ltd.
- DAFoam comes with the OpenFOAM® software however, it can also be separately downloaded and installed from
  - https://www.openfoam.com/

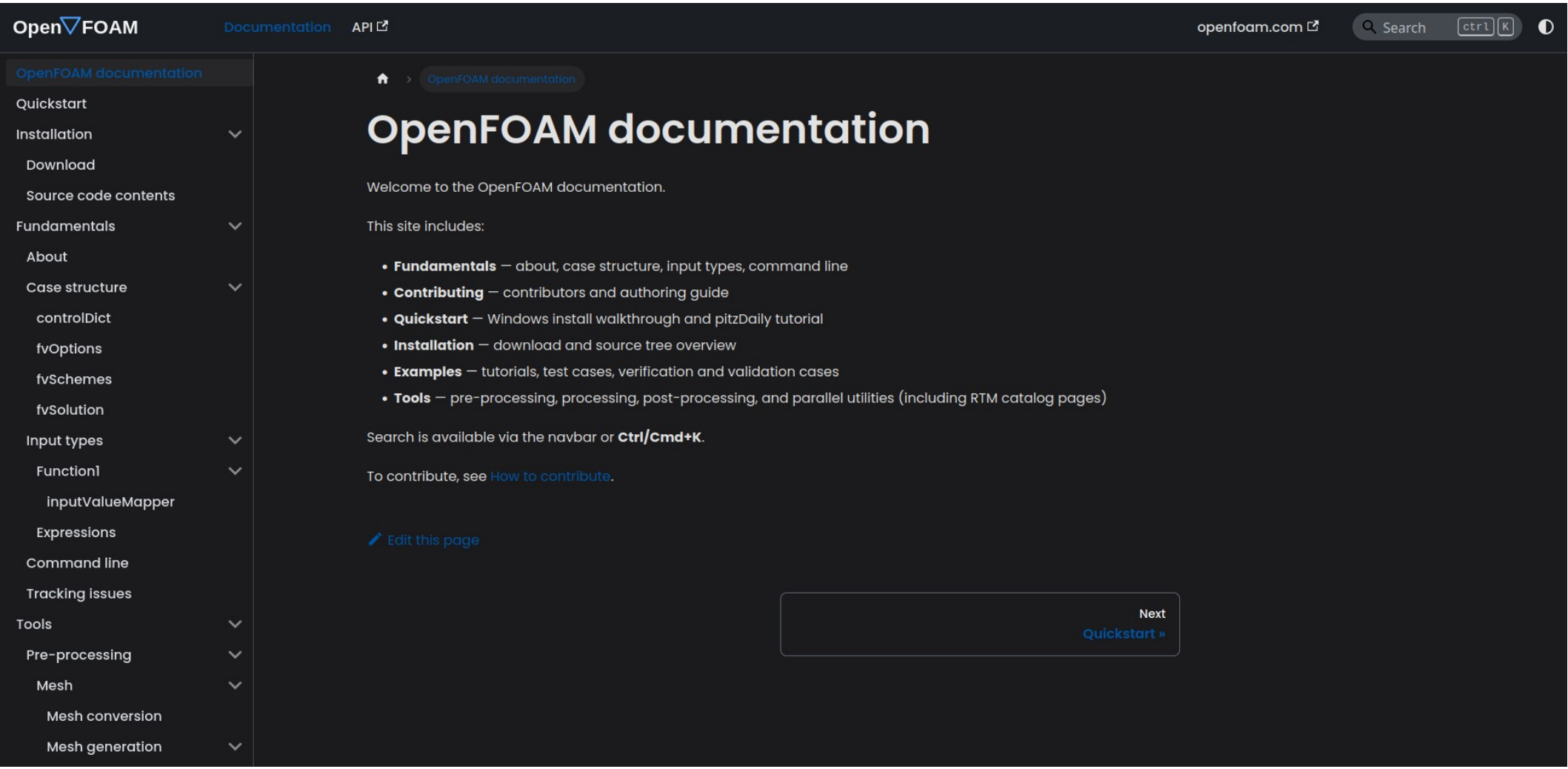


Available on GitLab



User Guide

- Can be accessed from the “Documentation” link on the web site



Tutorials

```
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/
emrealpman$ ls
Allclean     compressible      heatTransfer    multiphase
Allcollect   discreteMethods   incompressible  preProcessing
Allrun       DNS               IO              resources
Alltest      electromagnetics  lagrangian      stressAnalysis
basic        financial         mesh            verificationAndValidation
combustion   finiteArea        modules
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/
emrealpman$
```

Tutorials

```
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/
emrealpman$ ls
Allclean     compressible      heatTransfer    multiphase
Allcollect   discreteMethods   incompressible  preProcessing
Allrun       DNS               IO              resources
Alltest      electromagnetics  lagrangian      stressAnalysis
basic        financial         mesh            verificationAndValidation
combustion   finiteArea        modules
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/
emrealpman$ cd incompressible/
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/incompressible/
emrealpman$ ls
adjointOptimisationFoam       nonNewtonianIcoFoam  porousSimpleFoam
adjointShapeOptimizationFoam  overPimpleDyMFoam    shallowWaterFoam
boundaryFoam                  overSimpleFoam       simpleFoam
icoFoam                       pimpleFoam           SRFPimpleFoam
lumpedPointMotion             pisoFoam             SRFSimpleFoam
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/incompressible/
emrealpman$
```

Each folder corresponds to a solver for incompressible flow

Tutorials

```
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/incompressible/
emrealpman$ ls
adjointOptimisationFoam       nonNewtonianIcoFoam  porousSimpleFoam
adjointShapeOptimizationFoam  overPimpleDyMFoam    shallowWaterFoam
boundaryFoam                  overSimpleFoam       simpleFoam
icoFoam                       pimpleFoam           SRFPimpleFoam
lumpedPointMotion             pisoFoam             SRFSimpleFoam
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/incompressible/
emrealpman$ cd pimpleFoam/
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/incompressible/pimpleFoam/
emrealpman$ ls
laminar  LES  RAS
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/incompressible/pimpleFoam/
emrealpman$ cd RAS/
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/incompressible/pimpleFoam/
RAS/
emrealpman$ ls
axialTurbine_rotating_oneBlade  rotatingFanInRoom
ellipsekkLOmega                 TJunction
oscillatingInletACMI2D          TJunctionArrheniusBirdCarreauTransport
oscillatingInletPeriodicAMI2D   TJunctionFan
pitzDaily                       TJunctionSwitching
propeller                       wingMotion
```

OpenFOAM® cases which use pimpleFoam solver

Tutorials

```
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/incompressible/pimpleFoam/
RAS/
emrealpman$ cd propeller/
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/incompressible/pimpleFoam/
RAS/propeller/
emrealpman$ ls
0.orig  Allclean  Allrun  Allrun.pre  constant  system
openfoam2412:/usr/lib/openfoam/openfoam2412/tutorials/incompressible/pimpleFoam/
RAS/propeller/
emrealpman$
```

The *0* folder may appear as *0.orig*. This is typically done to keep the original settings

The files *Allclean*, *Allrun*, *Allrun.pre* are Linux scripts. They can be executed to run the program.

The *Allclean* scripts erases the files created during the solution for a fresh start.

Necessary Folders

- **0**: Contains initial and boundary conditions for the flow variables (pressure, velocity, turbulent quantities, etc...)
- **constant**: Contains fluid properties, turbulence properties, mesh information, mesh deformation information (for dynamic meshes, ...)
- **system**: Numerical schemes, solution algorithms, mesh generation controls, solution controls, ...

Allrun script

```
#!/bin/sh
cd "${0%/*}" || exit                                # Run from this directory
. ${WM_PROJECT_DIR:?}/bin/tools/RunFunctions        # Tutorial run functions
#------------------------------------------------------------------------------

if notTest "$@"
then

    ./Allrun.pre

    restore0Dir

    runApplication decomposePar

    runParallel $(getApplication)

    runApplication reconstructPar

fi

#------------------------------------------------------------------------------
```

Allrun.pre script

```
#!/bin/sh
cd "${0%/*}" || exit                                # Run from this directory
. ${WM_PROJECT_DIR:?}/bin/tools/RunFunctions        # Tutorial run functions
#------------------------------------------------------------------------------

# copy propeller surface from resources directory
cp -rf \
    "$FOAM_TUTORIALS"/resources/geometry/propeller \
    constant/triSurface

runApplication blockMesh

runApplication surfaceFeatureExtract

runApplication snappyHexMesh -overwrite

runApplication renumberMesh -overwrite

# force removal of fields generated by snappy
rm -rf 0
```

DAFoam Example

- NACA 0012 airfoil shape optimization for transonic flow
- Go to the corresponding working folder and open a “Terminal”.
- Execute the command:
  - *docker run -it --rm -u dafoamuser --mount "type=bind,src=.,target=/home/dafoamuser/mount" -w /home/dafoamuser/mount dafoam/opt-packages:v5.1.0 bash*
- This is going to create a virtual machine.
- *dafoamuser* will be your username on this machine

DAFoam Example

- Folder ownership may need to be changed. Otherwise files and folders will be read-only.

```
emrealpman@eagle:~/OpenFOAM/tutorials-main/NACA0012_Airfoil$ docker run -it --rm
 -u dafoamuser --mount "type=bind,src=.,target=/home/dafoamuser/mount" -w /home/
dafoamuser/mount dafoam/opt-packages:v5.1.0 bash
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

dafoamuser@f5006fcfc72c:~/mount$ ls -l
total 16
drwxrwxr-x 8 ubuntu ubuntu 4096 Sep  2 08:23 incompressible
drwxrwxr-x 7 ubuntu ubuntu 4096 Sep 22  2022 multipoint
drwxrwxr-x 7 ubuntu ubuntu 4096 Sep 22  2022 subsonic
drwxrwxr-x 7 ubuntu ubuntu 4096 Sep 22  2022 transonic
dafoamuser@f5006fcfc72c:~/mount$
```

DAFoam Example

- The super user password for *dafoamuser* is *dafoamuser*.

```
dafoamuser@f5006fcfc72c:~/mount$ ls -l
total 16
drwxrwxr-x 8 ubuntu ubuntu 4096 Sep  2 08:23 incompressible
drwxrwxr-x 7 ubuntu ubuntu 4096 Sep 22  2022 multipoint
drwxrwxr-x 7 ubuntu ubuntu 4096 Sep 22  2022 subsonic
drwxrwxr-x 7 ubuntu ubuntu 4096 Sep 22  2022 transonic
dafoamuser@f5006fcfc72c:~/mount$ sudo chown dafoamuser:dafoamuser *
[sudo] password for dafoamuser:
dafoamuser@f5006fcfc72c:~/mount$ ls -l
total 16
drwxrwxr-x 8 dafoamuser dafoamuser 4096 Sep  2 08:23 incompressible
drwxrwxr-x 7 dafoamuser dafoamuser 4096 Sep 22  2022 multipoint
drwxrwxr-x 7 dafoamuser dafoamuser 4096 Sep 22  2022 subsonic
drwxrwxr-x 7 dafoamuser dafoamuser 4096 Sep 22  2022 transonic
dafoamuser@f5006fcfc72c:~/mount$
```

DAFoam Example

- Transonic folder:
  - Please check the file and folder ownership
- Problem is solved by running the *preProcessing.sh* and *runScript.py* commands, respectively.

```
dafoamuser@4d5781eaaca8:~/mount$ ls -l
total 1184
drwxrwxr-x 2 dafoamuser dafoamuser   4096 Jul 31 21:12 0.orig
-rwxrwxr-x 1 dafoamuser dafoamuser    611 Jul 31 21:12 Allclean.sh
drwxrwxr-x 2 dafoamuser dafoamuser   4096 Jul 31 21:12 FFD
-rw-r--r-- 1 dafoamuser dafoamuser 172032 Sep  2 14:54 OptView.hst
drwxrwxr-x 2 dafoamuser dafoamuser   4096 Sep  3 07:52 constant
-rwxrwxr-x 1 dafoamuser dafoamuser   7644 Jul 31 21:12 genAirFoilMesh.py
-rw-r--r-- 1 dafoamuser dafoamuser 981414 Sep  2 14:34 mphys.html
-rwxrwxr-x 1 dafoamuser dafoamuser      0 Jul 31 21:12 paraview.foam
-rwxrwxr-x 1 dafoamuser dafoamuser    541 Jul 31 21:12 preProcessing.sh
drwxrwxr-x 2 dafoamuser dafoamuser   4096 Jul 31 21:12 profiles
-rwxrwxr-x 1 dafoamuser dafoamuser  10509 Jul 31 21:12 runScript.py
drwxr-xr-x 3 dafoamuser dafoamuser   4096 Sep  2 14:34 runScript_out
drwxrwxr-x 2 dafoamuser dafoamuser   4096 Jul 31 21:12 system
dafoamuser@4d5781eaaca8:~/mount$
```

Optimizers

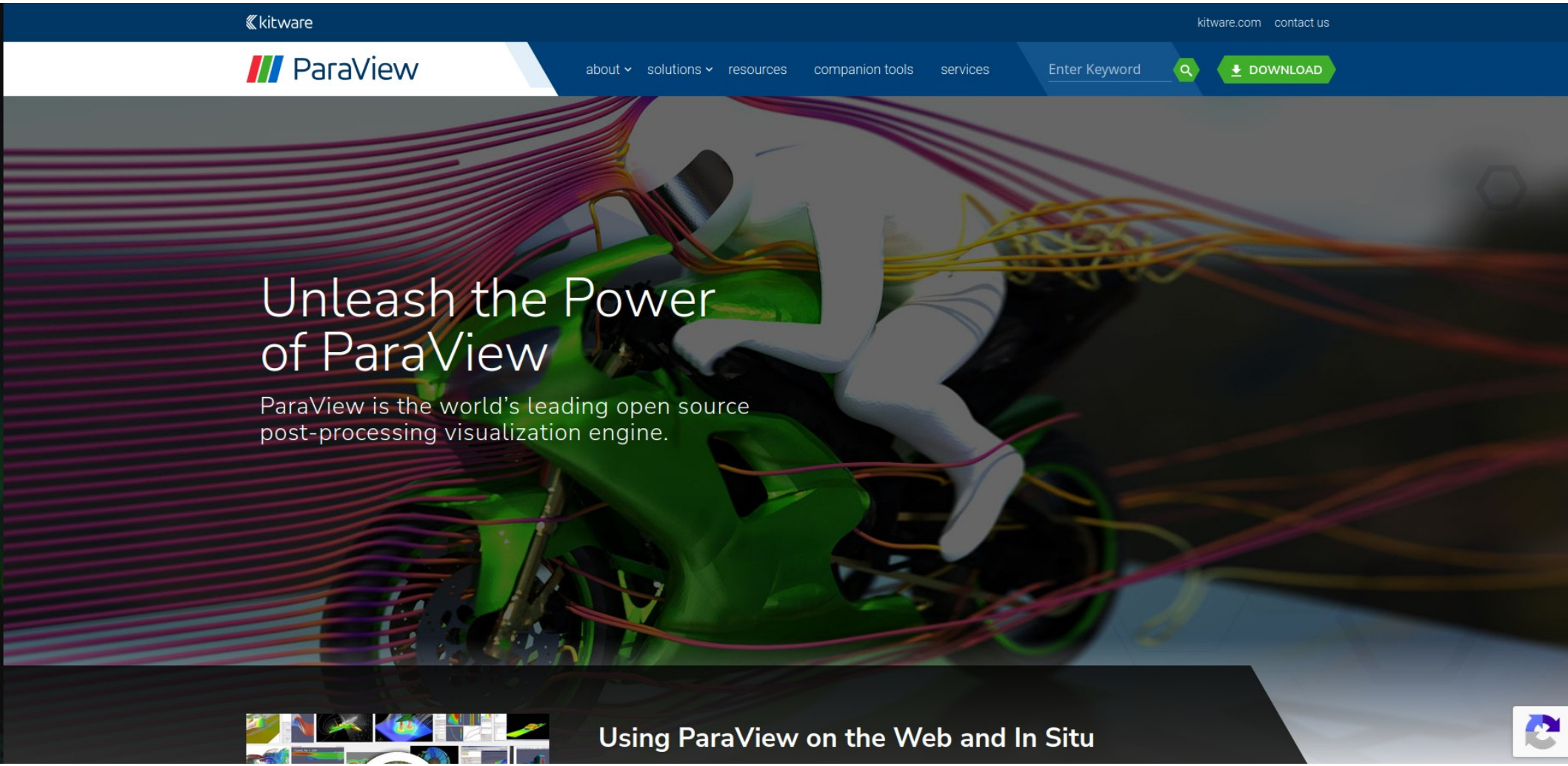
- Please refer to
  - https://mdolab-pyoptsparse.readthedocs-hosted.com/en/latest/

Final Design

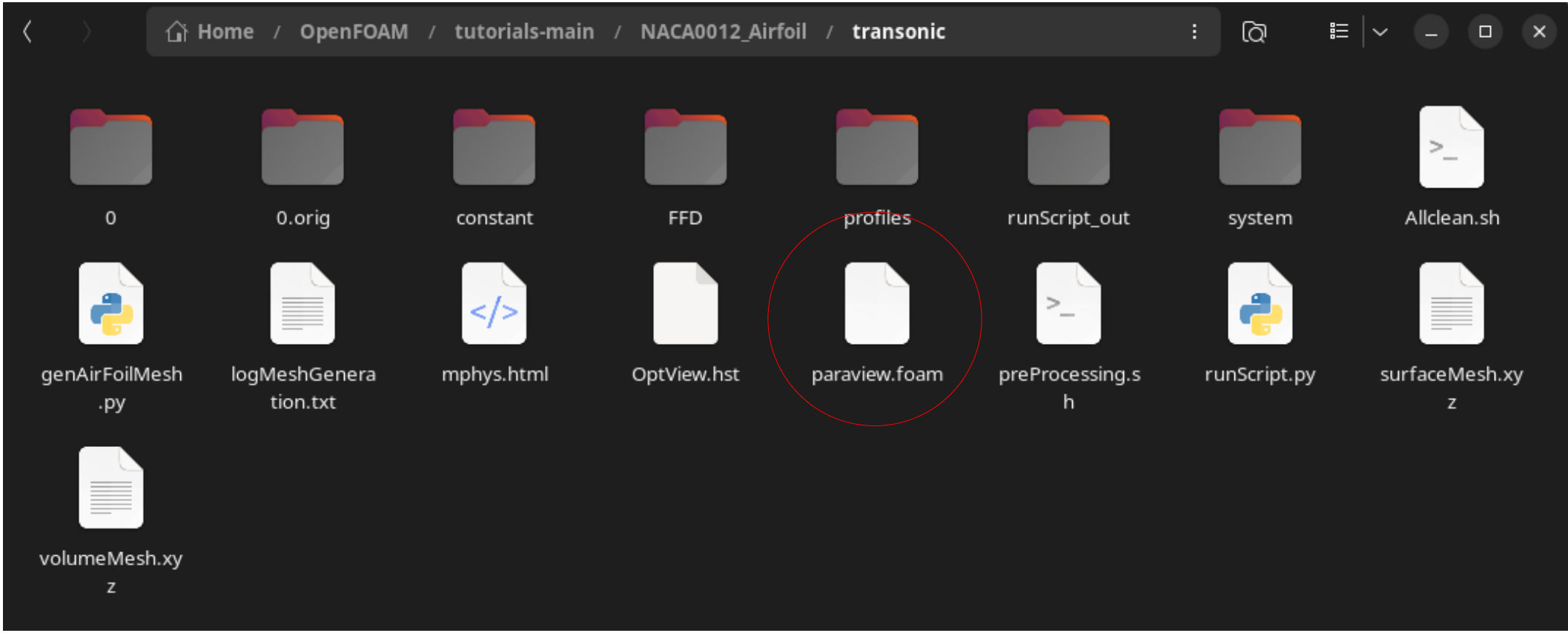
```
Nonlinear constraints
{'geometry.rcon': array([0.79999919, 0.79999919]),
 'geometry.thickcon': array([0.89153056, 0.94615066, 0.99874647, 1.04409286, 1.0
7153417,
       1.06717403, 1.01077188, 0.89507549, 0.73365887, 0.54070854,
       0.89153056, 0.94615066, 0.99874647, 1.04409286, 1.07153417,
       1.06717403, 1.01077188, 0.89507549, 0.73365887, 0.54070854]),
 'geometry.volcon': array([1.]),
 'scenario1.aero_post.CL': array([0.49994982])}

Objectives
{'scenario1.aero_post.CD': array([0.01277527])}

dafoamuser@4d5781eaaca8:~/mount$
```

Post Processing

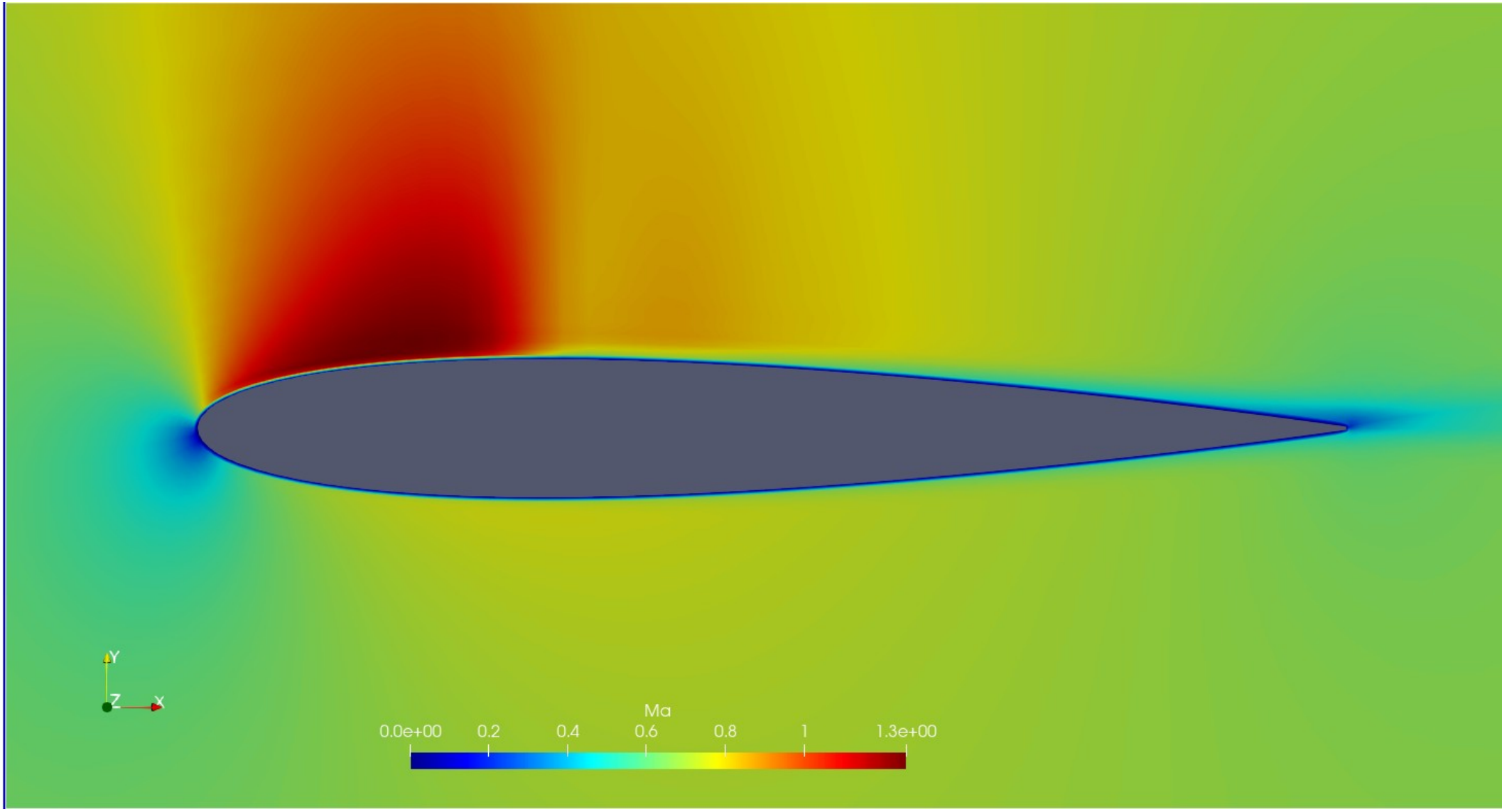
- Visualization of the results can be done using the ParaView software



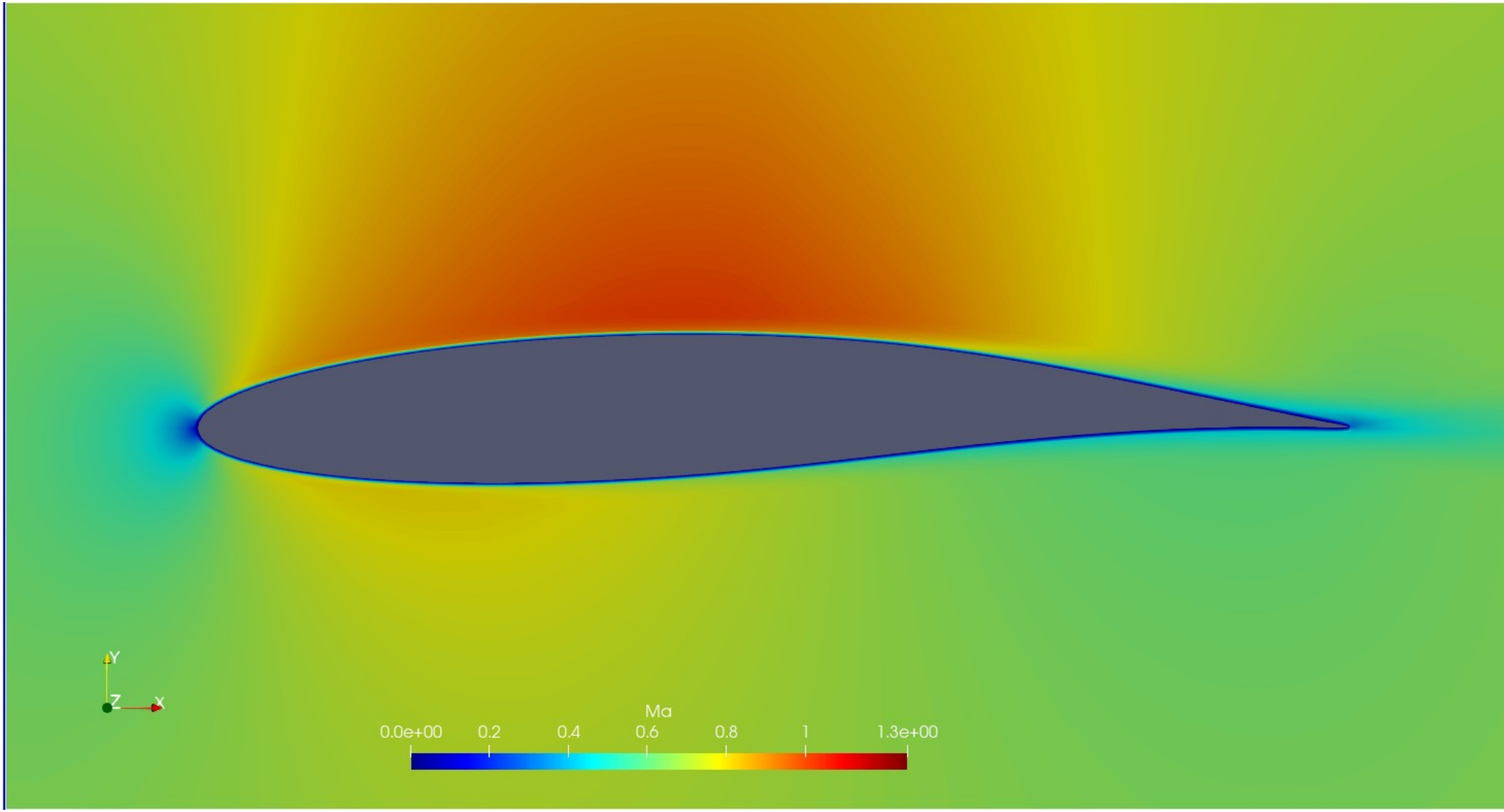
Post Processing



Initial Geometry



Final Geometry



Topology Optimization

- Mathematical tool which tries to obtain optimum material distribution in a given space under applied loads.
  - Minimize the weight of an object without reducing its load carrying capacity.
    - Same mechanical strength at minimum weight
  - Obtain the optimum layout of cooling channels in a heat exchanger
    - Optimum cooling performance at minimum pressure loss
  - ...

Bendsøe, M. P., & Sigmund, O. (2004). Topology optimization: theory, methods, and applications (Vol. 2). Berlin: Springer.

Topology Optimization Problem

- A single objective optimization problem aims to minimize (or maximize) a scalar objective function;

$$minimize\, f\left(\vec{\gamma}, \vec{w}\right)$$
$$st\,; \vec{R}\left(\vec{\gamma}, \vec{w}\right) = 0$$
$$\vec{g}\left(\vec{\gamma}\right) < 0$$
$$0 < \vec{\gamma} < 1$$

Effect of material distribution on the governing equations is typically included via a penalty function

- Where:
  - f is the objective function
  - $\vec{\gamma}$ material distribution, (1 for fluid, 0 for solid)
  - $\vec{g}\left(\vec{\gamma}\right)$ inequality constraint functions (e.g. minimum or maximum fluid volume)

Example

- Optimum cooling channel distribution for a heat sink.
- A rectangular block filled with liquid which is initially at 300 K. The upper wall of the block is kept at 340 K while the lower wall of the block is kept at 310 K.
- The aim is to adjust the flow within this block such that the exit temperature of the cooling liquid will be maximum at the minimum possible pressure loss.

Shape Optimization vs. Topology Optimization

- Topology optimization aims to obtain optimum material layout in a domain.
  - It does not require an initial design to start from.
- Shape optimization aims to get an optimum set of the parameters used to construct a geometry.
  - An existing design is optimized